



**Univerzitet u Novom Sadu
Fakultet tehničkih nauka**



**UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
NOVI SAD**

ISPITNI RAD

Kandidat: Bogdan Ljubinković
Broj indeksa: SV2/2023

Predmet: Objektno orijentisano programiranje 2
Tema rada: Robot u lavirintu u Knososu

Mentor rada: dr Dušan Kenjić

Novi Sad, decembar, 2024.

Sadržaj

1 Uvod.....	3
1.1 Kratko objašnjenje problema I cilj projekta.....	3
1.2 Opis igre I pravila.....	4
2 Rad U/I podсистema.....	5
2.1 Proces kompajliranja.....	5
2.2 Učitavanje podataka (input).....	6
2.3 Interakcija sa programom.....	7
2.4 Upis podataka (output).....	7
3 Struktura projekta I pregled fajlova.....	8
3.1 Pregled fajlova.....	8
3.2 Pregled klasa.....	9
4 Tehnička analiza programa.....	18
4.1 Ulazna tačka programa – main funkcija.....	18
4.2 Pokretanje igre – funkcija Igra::pokreniIgru().....	21
4.3 Generisanje lavirinta.....	22
4.4 Algoritam generisanja lavirinta.....	23
4.5 Struktura ivica I disjunktni skup.....	23
4.6 Spajanje disjunktnih skupova.....	24
4.7 Kretanje igrača – funkcija Igra::pomeriRobota(...).....	25
4.8 Kretanje minotaura.....	26
4.9 Upis stanja.....	26
4.10 Završi igru I oslobodi resurse.....	27
4.11 Nasledjivanje I polimorfizam.....	27
5 Testiranje vremena generisanja.....	29
6 Izlazni rezultati – fajl lavirint.txt.....	31

1 Uvod

1.1 Kratko objašnjenje problema I cilj projekta

Problem koji se obrađuje u ovom projektu zasnovan je na izazovima navigacije kroz lavirint, inspirisan mitskim Labirintom u Knososu, koji je bio ozloglašena građevina mionske civilizacije. Ključni izazov leži u programiranju sistema koji omogućava robotskom entitetu da se uspešno kreće kroz lavirint, pronalazeći izlaz uz izbegavanje prepreka, ćorsokaka i opasnosti u vidu mitskog bića Minotaura.

Cilj ovog projekta je da se kroz implementaciju algoritama, objektnog programiranja i principa nasleđivanja razvije igra koja simulira problem navigacije kroz lavirint. U ovoj simulaciji, robot mora:

- Efikasno koristiti resurse (predmete sa specijalnim efektima) da savlada izazove.
- Navigirati kroz dinamički generisani lavirint uz poštovanje pravila igre.
- Izbeći ili savladati Minotaura koristeći dodatke kao što su mač ili štit.

Projekat ne samo da zahteva tehničko razumevanje algoritama, već se fokusira i na praktičnu implementaciju kroz C++ jezik, uz primenu savremenih koncepata kao što su nasleđivanje, polimorfizam i modularnost koda. Kroz ovu igru, korisnici će steći praktično iskustvo u primeni naprednih tehnika programiranja, kao i u dizajnu složenih sistema simulacije.

1.2 Opis igre I pravila

Igra "Robot u lavirintu u Knososu" simulira kretanje robota kroz dinamički generisani lavirint, pri čemu igrač preuzima ulogu kontrolisanja robota sa ciljem da pronade izlaz. Lavirint se sastoji od niza elemenata, uključujući zidove, prolaze, ulaz, izlaz, predmete sa specijalnim efektima, kao i protivnika – Minotaura.

Lavirint:

- Lavirint je generisan kao matrica čija su dimenzije zadate kao argumenti komandne linije.
- Elementi lavirinta uključuju:
 - # - Zidovi: neprolazni elementi koji formiraju strukturu lavirinta.
 - . (' ') - Prolazi: dostupni putevi za kretanje robota i Minotaura.
 - U - Ulaz: početna tačka robota, nasumično postavljena u prvom redu.
 - I - Izlaz: cilj igre, nasumično postavljen u poslednjem redu.
 - P - Predmeti: specijalni efekti koji pomažu robotu u rešavanju lavirinta.
 - M - Minotaur: protivnik koji može uništiti robota.

Robot:

- Robot se nalazi na poziciji neposredno ispod ulaza (U) pri početku igre.
- Korisnik kontroliše robota pomoću unosa komandi za kretanje:
 - **Gore, dole, levo, desno** (definisane tasterske komande).
- Robot ne može proći kroz zidove (#).
- Ako robot naiđe na predmet (P), aktivira se specijalni efekat sa trajanjem od 3 poteza.

Minotaur:

- Minotaur se nasumično kreće po lavirintu, ali ne može proći kroz zidove (#).
- Ako robot dođe na jedno polje udaljenosti od Minotaura, Minotaur postaje agresivan i pokušava da ga uništi.
- Ukoliko Minotaur stane na polje sa predmetom (P), predmet se uništava.

Predmeti i efekti:

- **Magla rata:** Ograničava vidljivost robota na oblast dimenzija 3x3.
- **Mač:** Omogućava robotu da uništi Minotaura.
- **Štit:** Pruža zaštitu od napada Minotaura.
- **Čekić:** Dozvoljava robotu da prolazi kroz zidove (#).

Cilj igre:

- Pronalaženje izlaza (I) uz izbegavanje ili savladavanje prepreka.
- Igra se završava:
 - Dolaskom robota na polje izlaza (I).
 - Uništenjem robota od strane Minotaura.
 - Ručnim izlaskom iz igre komandom „Q“.

2 Rad U/I podсистema

2.1 Proces kompajliranja

Za početak neophodno je otvoriti terminal I kompajlirati lavirint_utils.cpp u objektni fajl (u mom slučaju koristimo g++ kompajler):

```
g++ -c lavirint_utils.cpp -o lavirint_utils.o
```

Ova komanda kompajlira lavirint_utils.cpp u objektni fajl lavirint_utils.o. Opcija -c znači “compile only” – ne pravi izvršni fajl, već samo objektni fajl. Sledeći korak je kreiranje statičke biblioteke liblavirint_utils.a. Po zahtevu projekta neophodno je logiku za kreiranje lavirinta odvojiti u posebnu biblioteku. Ja sam se opredelio za statičku. Unosimo u terminal:

```
ar rcs liblavirint_utils.a lavirint_utils.o
```

Koristimo alat ar za kreiranje statičke biblioteke liblavirint_utils.a iz objektnog fajla lavirint_utils.o. Opcija r – dodaj ili zameni fajl u biblioteci, opcija c – kreiraj biblioteku ako ne postoji I opcija s – dodaj indeks biblioteci za brži pristup simbolima. Naredni korak je kompajliranje glavnog programa sa statičkom bibliotekom:

```
g++ main.cpp -o main_program liblavirint_utils.a lavirint.cpp igra.cpp  
element.cpp minotaur.cpp predmeti.cpp robot.cpp upis_stanja.cpp -L. -  
llavirint_utils
```

Glavni fajl main.cpp se kompajlira zajedno sa svim navedenim .cpp fajlovima I statičkom bibliotekom liblavirint_utils.a. Opcija -o main_program specifikuje naziv izvršnog fajla na main_program. Opcija -L dodaje trenutni direktorijum (.) kao lokaciju gde se traže biblioteke, a opcija -llavirint_utils povezuje biblioteku liblavirint_utils.a gde se prefiks lib I ekstenzija .a automatski dodaju. Poslednji korak je pokretanje programa:

```
./main_program <broj_redova> <broj_kolona> <broj_predmeta>
```

Bitno je uneti isključivo celobrojne vrednosti tipa int, u suprotnom dolazi do izuzetaka koji će kasnije biti spomenuti.

2.2 Učitavanje podataka (input)

Putem komandne linije neophodno je uneti dimenzije lavirinta I broj predmeta (dodatna 3 argumenta posle ./main_program). Na primer:

```
./main_program 21 21 5
```

Ovi podaci se prihvataju unutar main funkcije kao parametri. Main funkcija proverava njihovu validnost. Ako neka od vrednosti nije validna, izvršenje programa se završava vraćanjem povratne vrednosti 1 I ispisom greške.

NAPOMENA: Ako je broj redova ili broj kolona paran broj, main funkcija će povećati za jedan kako bi imali neparan broj. Razlog toga je principa rada **modifikovanog Kruskalovog algoritma** koji se kasnije koristi za generisanje lavirinta. Kako on radi isključivo sa neparnim vrednostima kako bi pravilno generisao lavirint, ovo je za sada jedino poznato ograničenje u programu koje sam otkrio.

Nakon ispravnog unosa podataka, pokreće sa program sa kratkom porukom o vremenu koje je bilo potrebno za generisanje lavirinta, na primer:

```
Lavirint generisan za 5.2988e-05 sekundi
```

Što je oko 0.000052988 sekundi za lavirint dimenzija 21x21. Red ispod počinje odbrojavanje tajmera od 3 sekunde kako bi se korisnik psihički pripremio za igru, a I kako bi pre svega imao priliku da pogleda koliko je vremena bilo potrebno za generisanje lavirinta. Nakon toga prikazuje se lavirint.

2.3 Interakcija sa programom

Nakon obrade podataka, unutar main funkcije se instancira objekta klase Igra kome se prosledjuju atributi broj_redova, broj_kolona I broj_predmeta. Nakon poziva metode za početak igre, cela interakcija se svodi na unošenje komandi w, a, s I d (ili W, A, S, D) za kretanje robota, ili unos karaktera q (Q) za prekid igre. Nakon pobede robota, podebe minotaura ili unosa komande za prekid igre, igra se završava I poziva se funkcija za oslobađanje resursa. To bi naravno bilo najprostije objašnjeno, kasnije ulazimo u detalje nakon navodjenja svih klasa I njihovih najvažnijih metoda I atributa.

2.4 Upis podataka (output)

Program prikazuje stanje igre u realnom vremenu u konzoli (lavirint, pozicije robota, minotaura, predmeta I aktivne efekte). Ispis u konzoli se stalno osvežava čišćenjem terminala pozivom komande:

```
system("clear") // ili system("cls") za Windows
```

Tako se stalno čisti memorija terminala I estetski mnogo lepše izgleda, usput I ono verovatno najbitnije, mnogo je lakše pratiti tok igre kada je lavirint fiksiran na jednom mestu. Ova komanda radi za Linux I MacOS platforme, za Windows neophodno je promeniti na `system("cls")`. Takodje program koristi razne predefinisane boje, naravno iz estetskih razloga.

Nakon završetka igre, po zahtevu projekta, neophodno je upisati poslednje stanje igre I poruku u neku izlaznu datoteku. Program koristi .txt datoteku (lavirint.txt). Tada se pored matrice lavirinta prikazuju jedna od 3 poruke o pobedi robota, minotaura ili prekidu igre (Q). Logika za upis se nalazi u odvojenoj klasi UpisStanja koju koristimo kao statička. Metoda ove klase se poziva na kraju metode Igra::pokreniIgru()).

Primer lavirinta dimenzija 16x31 sa 5 predmeta može se videti na slici ispod (koristio sam ‘ ‘ umesto ‘.’ zato što opet estetski izgleda mnogo lepše, na funkcionisanje programa apsolutno ništa ne utiče).

```
#####U#####
#      # R      P      # #      P  #
#  ###  # #####  # #####  ###  #####
#      #      #      # #      #
# #####  ###  # #####  #####
#P  # # # # # # #      #
###  # ###  # ###  ###  #####  #
# # #      # # #      M#  # # #      #
# #####  # # ###  ###  # # #####  #
#      #      P#      #      # #      #
# # ###  ###  # #####  ###  # # # #
# #      #      #      #      # # #
###  ###  # # # #####  #####  #
#      # # # # #      #      P  # #
#####I#####
```

Primer generisanog lavirinta (runda 1)

3 Struktura projekta I pregled fajlova

3.1 Pregled fajlova

- `main.cpp` – deklaracija I definicija funkcije `main`.
- `element.hpp` – deklaracija klase `Element` koja predstavlja osnovni element lavirinta. Svaki objekat u lavirintu je ili `Element` ili naslednik klase `Element`.
- `element.cpp` – ovde se implementiraju sve metode klase `Element`.
- `robot.hpp` – deklariramo klasu `Robot`. Klasa `Robot` predstavlja robota u lavirintu, nasledjuje klasu `Element`.
- `robot.cpp` – ovde se implementiraju sve metode klase `Robot`
- `minotaur.hpp` – deklaracija klase `Minotaur` koja predstavlja minotaura u lavirintu. Takodje nasledjuje klasu `element` I za sada nema specifičnih metoda ili atributa
- `minotaur.cpp` – predstavlja implementaciju svih metoda klase `Minotaur`.
- `predmeti.hpp` – predstavlja deklaraciju klase `Predmet` u lavirintu, koja takodje nasledjuje klasu `Element`.
- `predmeti.cpp` – implementacija svih metoda klase `Predmet`
- `lavirint.hpp` – unutar ovog fajla nalazi se deklaracija klase `Lavirint`
- `lavirint.cpp` – implementacija svih metoda klase `lavirint`.
- `lavirint_utils.hpp` – deklaracija funkcija za generisanje lavirinta, postavljanje robota, minotaura I predmeta. Koristimo kasnije kao statičku biblioteku.
- `lavirint_utils.cpp` – implementacija funkcija za generisanje čavirinta, postavljanje robota, minotaura I predmeta.
- `igra.hpp` – deklaracija klase koja predstavlja igru. Ovde se nalazi najveći deo logike funkcionisanja programa.
- `igra.cpp` – implementacija svih metoda klase `Igra`

- upis_stanja.hpp – deklaracija klase UpisStanja koja omogućava upis stanja igre u fajl. U fajl upisujemo matricu lavirinta I ishod igre.
- upis_stanja.cpp – implementacija metoda klase upis stanja

3.2 Pregled klasa

Element

Klasa Element predstavlja osnovni element lavirinta. Svaki objekat u lavirintu je ili Element ili naslednik klase Element. Prolaz ‘ ‘ I zidovi ‘#’ su takodje klase element. Klase Element naslednju klase Predmet, Minotaur I Robot.

Sadrži samo jedan protected atribut tipa char a to je simbol elementa. Od public ima parametrizovan I default konstruktor, virtuelni destruktor, setter I getter za simbol, kao I virtuelnu metodu kloniraj() koja se overrajduje prilikom nasledjivanja (povratna vrednost tipa Element*).

```
class Element {
protected:
    char simbol;
public:
    Element(char simbol);
    Element() = default;
    virtual ~Element();
    char getSimbol() const;
    void setSimbol(char simbol);

    virtual Element* kloniraj() const;
};
```

Klasa Element koja je deklarirana unutar fajla element.hpp

Robot

Klasa Robot predstavlja naravno robota u lavirintu, entitet kojeg naš igrač kontroliše. Nasledjuje klasu Element. Jedina razlika je overrajdovana metoda kloniraj() koja sada ima tip povratne vrednosti Robot*. Moguća je kasnija nadogradnja klase, za sada nije više bilo potrebno.

You, 4 days ago | 1 author (You)

```
class Robot : public Element {
public:
    Robot(char simbol);
    ~Robot();

    char getSimbol() const;
    void setSimbol(char simbol);

    Robot* kloniraj() const override;
};
```

Klasa Robot deklarirana unutar fajla robot.hpp

Minotaur

Klasa Minotaur predstavlja minotaura u lavirintu. Takođe nasledjuje klasu element I nema specifičnih atributa ili metoda različitih od onih koji su već nasledjeni. Ima parametrizovan konstruktor, destruktor I overrajdovanu metodu kloniraj koja vraća Minotaur*.

You, yesterday | 1 author (You)

```
class Minotaur : public Element {
public:
    Minotaur(char simbol);
    ~Minotaur();

    Minotaur* kloniraj() const override;
};
```

Klasa Minotaur deklarirana unutar fajla minotaur.hpp

Predmet

Klasa Predmet predstavlja predmet u lavirintu. Nasledjuje klasu Element. Kao što je već navedeno u uvodu, predmeti su označeni sa 'P' I imaju specijalne efekte poput 'Magla rata' koja smanjuje vidljivost matrice na 3x3, 'Čekić' koji ima mogućnost da uništi zid, 'Mač' koji može da uništi minotaura I 'Štit' koji ima sposobnost odbrane od minotaura.

Za razliku od Minotaur I Robot klasa, Predmet ima još jedan atribut koji nije nasledjen, a to je std::string efekat. Prilikom generisanja lavirinta pomoću biblioteke lavirint_utils metodom LavirintUtils::postaviPredmete(...), postavljaju se predmeti I njihovi specijalni efekti random. Klasa Predmet ima parametrizovan konstruktor sa 2 parametra, simbol I efekat.

You, 5 days ago | 1 author (You)

```
class Predmet : public Element {
private:
    std::string efekat;
public:
    Predmet(char simbol, std::string efekat);
    ~Predmet();

    std::string getEfekat() const;
    void setEfekat(std::string efekat);
};
```

Klasa Predmet deklarirana unutar predmet.hpp

Lavirint

Sada prelazimo na malo složenije klase. Jasno je da klasa Lavirint predstavlja lavirint u našoj igri. Klasa Lavirint ima 3 privatna atributa, a to su brojRedova tipa int, brojKolona tipa int matricu tipa Element***. Svaki element matrice je pokazivač na objekat tipa Element, dinamički kreiran sa 'new'. Niz pokazivača čine jedan red Element**, a niz redova Element***.

Klasa ima jedan parametrizovan konstruktor koji prima broj redova I broj kolona, destruktora, metode za get I set metode, metodu getElement koja vraća Element objekat (ili njegove naslednike) po const referenci. Ima 2 parametra a to su koordinate x I y unutar matrice. Isto tako ima I metodu za postavljanje elementa na određenoj poziciji, a to je metoda setElement. Ima 3 parametra, x, y I pokazivač na objekat tipa Element, odnosno Element*.

Pored ovih metoda, klasa ima I metodu generisiLavirint koja ima jedan parametar a to je broj predmeta. Jedina njena uloga jeste da pozove statičku metodu iz klase LavirintUtils. O tome više reči kasnije. Sliku klase Lavirint možete videti na narednoj stranici dokumenta.

You, 5 days ago | 1 author (You)

```
class Lavirint {
private:
    int brojRedova;
    int brojKolona;
    Element*** matrica;
public:
    Lavirint(int brojRedova, int brojKolona);
    ~Lavirint();
    int getBrojRedova() const;
    int getBrojKolona() const;
    Element*** getMatrica() const;
    void setBrojRedova(int brojRedova);
    void setBrojKolona(int brojKolona);
    void setMatrica(Element*** matrica);

    Element& getElement(int x, int y) const;
    void setElement(int x, int y, Element* element);

    void generisiLavirint(int brojPredmeta);
};
```

Figure 1: Klasa Lavirint deklarirana unutar fajla lavirint.hpp

Igra

Klasa Igra je verovatno najkompleksnija klasa u smislu broja metoda. Ne može se reći ni za jednu klasu da je najbitnija, ali kada bih morao izdvojiti jednu, to bi sigurno bila Igra. U njoj se nalazi najveći deo logike za funkcionisanje naše igre.

Ima više privatnih atributa: Lavirint* lavirint, što je pokazivač na element klase Lavirint, brojPredmeta tipa int, bool krajIgre kao indikator kraja igre, bool robotPobedio kao indikator pobede robota, bool minotaurPobedio kao indikator pobede minotaura, bool minotaurZiv kao indikator da li je živ minotaur (nakon svakog poteza robota generiše se random potez minotaura, koji naravno nije potreban ako je minotaur mrtav), trajanjeEfekta tipa int kao indikator koliko rundi je preostalo za trajanje efekta. Pored ovih atributa imamo još std::string trenutniEfekat koji pokazuje koji efekat je trenutno aktivan (‘’ ako nema efekta) I po 2 atributa (4 ukupno) za x I y koordinate robota I minotaura.

Klasa Igra je centralna tačka logike naše igre i obuhvata ključne metode za upravljanje njenim tokom, interakciju između robota, lavirinta i Minotaura, kao i kontrolu stanja igre. Javne metode klase omogućavaju rad sa privatnim atributima, kontrolu toka igre i prikaz stanja lavirinta.

Konstruktor Igra(int brojRedova, int brojKolona, int brojPredmeta) inicijalizuje lavirint sa zadatim dimenzijama i brojem predmeta, kreirajući sve potrebne elemente za početak igre. Destruktor ~Igra() osigurava pravilno oslobađanje memorije i zatvaranje igre, sprečavajući curenje resursa.

Metoda pokreniIgru() sadrži glavnu petlju igre koja kontroliše sve aspekte njenog toka, uključujući poteze robota i Minotaura, interakcije sa lavirintom, aktivaciju predmeta i proveru pobednika. Metoda završiIgru() završava igru pozivanjem destruktora klase Lavirint.

Klasa pruža pristup osnovnim atributima kroz metode poput getLavirint() koja vraća pokazivač na lavirint, i getBrojPredmeta() koja vraća broj predmeta u igri. getKrajIgre() proverava da li je igra završena, dok getTrajanjeEfekta() vraća koliko poteza je preostalo za trenutni efekat predmeta. getTrenutniEfekat() vraća naziv trenutnog efekta ako postoji, ili prazan string ako nije aktivan. Tu su i metode getRobotX(), getRobotY(), getMinotaurX() i getMinotaurY() koje vraćaju trenutne koordinate robota i Minotaura.

Podešavanje atributa omogućeno je kroz metode poput setLavirint(Lavirint* lavirint), koja postavlja lavirint za igru, ili setBrojPredmeta(int brojPredmeta) za ažuriranje broja predmeta. Postavljanje indikatora za kraj igre obavlja se metodom setKrajIgre(bool krajIgre), dok setTrajanjeEfekta(int trajanjeEfekta) određuje trajanje efekta. Takođe, metode setRobotX(int robotX), setRobotY(int robotY), setMinotaurX(int minotaurX) i setMinotaurY(int minotaurY) postavljaju koordinate robota i Minotaura.

Metode za prikaz lavirinta uključuju prikaziLavirint(), koja ispisuje trenutni lavirint u konzoli, i prikaziLavirintZamagljen(), koja prikazuje lavirint sa ograničenom vidljivošću robota, što se koristi u efektu "Magla rata".

Kretanje likova u igri obuhvata metodu pomeriRobota(int x, int y), koja pomera robota na zadate koordinate ukoliko je potez validan, i metodu pomeriMinotauraRandom(), koja generiše nasumičan potez Minotaura.

Stanje igre procenjuje se metodom procenaStanja(), koja vraća status igre: 1 označava pobedu robota, 2 pobedu Minotaura, dok 3 označava prekid igre (npr. komandom "Q"). Nastavak igre označava se povratnom vrednošću 0.

```
class Igra {
private:
    Lavirint* lavirint;
    int brojPredmeta;
    bool krajIgre;
    bool robotPobedio;
    bool minotaurPobedio;
    bool minotaurZiv;
    int trajanjeEfekta;

    std::string trenutniEfekat;

    int robotX, robotY;
    int minotaurX, minotaurY;

public:
    Igra(int brojRedova, int brojKolona, int brojPredmeta);
    ~Igra();
    void pokreniIgru();
    void završiIgru();

    Lavirint* getLavirint() const;
    int getBrojPredmeta() const;
    bool getKrajIgre() const;
    int getTrajanjeEfekta() const;
    void setLavirint(Lavirint* lavirint);
    void setBrojPredmeta(int brojPredmeta);
    void setKrajIgre(bool krajIgre);
    void setTrajanjeEfekta(int trajanjeEfekta);
    std::string getTrenutniEfekat() const;
    void setTrenutniEfekat(std::string trenutniEfekat);

    void prikaziLavirint();

    int getRobotX() const;
    int getRobotY() const;
    int getMinotaurX() const;
    int getMinotaurY() const;
    void setRobotX(int robotX);
    void setRobotY(int robotY);
    void setMinotaurX(int minotaurX);
    void setMinotaurY(int minotaurY);

    void pomeriRobota(int x, int y);
    void pomeriMinotauraRandom();

    bool getRobotPobedio() const;
    bool getMinotaurPobedio() const;
    void setRobotPobedio(bool robotPobedio);
    void setMinotaurPobedio(bool minotaurPobedio);
    bool getMinotaurZiv() const;
    void setMinotaurZiv(bool minotaurStatus);

    int procenaStanja(); // 1 - robot pobedio, 2 - minotaur pobedio, 3 - prekid igre (Q), 0 - nastavak igre

    void porukaRobotPobedio();
    void porukaMinotaurPobedio();
    void porukaKrajIgre();

    void prikaziLavirintZamagljen();
};
```

Klasa Igra deklarirana unutar fajla igra.hpp

LavirintUtils

Klasa LavirintUtils je statička klasa koja sadrži funkcije za upravljanje lavirintom. Cilj ove klase je omogućavanje generisanja lavirinta i postavljanje ključnih elemenata igre, poput robota, Minotaura i predmeta. Kao statička klasa, koristi se isključivo za funkcionalnost bez potrebe za kreiranjem instanci.

Metoda generisiLavirint(Lavirint& lavirint, int brojPredmeta) generiše lavirint na osnovu zadanih dimenzija i broja predmeta (*). Kreira hodnike i zidove, osiguravajući povezivost svih delova lavirinta. Funkcija postaviRobota(Lavirint& lavirint) postavlja robota na početnu poziciju ispod ulaza (U) u lavirint, omogućavajući korisniku da započne igru sa jasno definisanim startnim položajem. Metoda postaviMinotaura(Lavirint& lavirint) nasumično pozicionira Minotaura u donjoj polovini lavirinta (**), proveravajući validnost pozicije kako bi se izbegle kolizije sa zidovima ili drugim elementima. Funkcija postaviPredmete(Lavirint& lavirint, int brojPredmeta) nasumično postavlja zadati broj predmeta u lavirintu. Predmeti imaju specijalne efekte, poput "Magla rata" ili "Štit", koji dodaju strategijsku dimenziju igri.

You, 6 days ago | 1 author (You)

```
class LavirintUtils {  
    public:  
        static void generisiLavirint(Lavirint& lavirint, int brojPredmeta);  
        static void postaviRobota(Lavirint& lavirint);  
        static void postaviMinotaura(Lavirint& lavirint);  
        static void postaviPredmete(Lavirint& lavirint, int brojPredmeta);  
};
```

Klasa LavirintUtils deklarirana unutar fajla lavirint_utils.hpp

(*) - ova metoda koristi modifikaciju Kruskalovog algoritma

(**) - prema postavci projekta minotaura treba postaviti nasumično unutar lavirinta. U ovom projektu se minotaur postavlja nasumično ali unutar donje polovine lavirinta. Za takvu odluku ima više razloga, jedan od njih je verovatno I najbitniji a to je opasnost da se minotaur ne postavi previše blizu igrača na samom početku igre. Zbog toga bi bilo velike šanse da igra bude jako kratka, što naravno nije zanimljivo. Drugo ako se minotaur postavi blizu samog početka, ima velike šanse da ga je lako zaobići I ostatak igre je praktično jako lagan gde on uopšte I nije prepreka. Pošto se postavlja na donju polovinu

lavrinta random, ovako mu dajemo više šanse da se pozicionira na neko zanimljivo mesto I zapravo napravi problem igraču. Kretanje minotaura je random I često se dešava da I nakon 5-6 rundi završi na istom mestu odakle je počeo.

DisjunktniSkup

Klasa DisjunktniSkup implementira strukturu disjunktних skupova (Union-Find) sa optimizacijama, kao što su kompresija puta i unija po rangu. Ova struktura je ključna za generisanje lavirinta jer omogućava proveru povezanosti i kreiranje hodnika.

Privatni atributi klase uključuju vektore roditelj i rang. Vektor roditelj čuva roditelja svakog elementa, omogućavajući brzo pronalaženje korena skupa, dok vektor rang čuva dubinu svakog skupa, što optimizuje spajanje.

Konstruktor DisjunktniSkup(int velicina) inicijalizuje strukturu za zadati broj elemenata, pri čemu je svaki element inicijalno u svom zasebnom skupu. Metoda pronadji(int x) vraća koren skupa kojem element pripada i implementira kompresiju puta, čime se ubrzavaju naredne operacije. Funkcija spojiSkupove(int x, int y) spaja dva skupa u jedan. Koristi rang za određivanje koji skup će postati podskup drugog, čime se smanjuje dubina stabla.

You, 7 days ago | 1 author (You)

```
class DisjunktniSkup {  
    private:  
        std::vector<int> roditelj;  
        std::vector<int> rang;  
    public:  
        DisjunktniSkup(int velicina);  
        int pronadji(int x);  
        bool spojiSkupove(int x, int y);  
};
```

Figure 2: Klasa DisjunktniSkup deklarirana unutar fajla lavirint_utils.hpp

UpisStanja

Klasa UpisStanja predstavlja jednostavnu, ali ključnu komponentu sistema za upravljanje stanjem igre. Njena glavna uloga je zapisivanje trenutnog stanja igre u fajl, čime se omogućava arhiviranje rezultata.

Klasa sadrži jednu javnu statičku metodu upisiUFajl(Lavirint& lavirint, int stanje). Ova metoda omogućava zapisivanje trenutnog stanja lavirinta u fajl, uključujući matricu lavirinta i ishod igre. Parametar Lavirint& lavirint predstavlja referencu na lavirint čije stanje treba zapisati, dok int stanje označava ishod igre, pri čemu različite vrednosti mogu označavati specifične situacije (npr. 1 za pobedu robota, 2 za pobedu Minotaura, ili 3 za prekid igre).

Metoda koristi standardnu biblioteku za rad sa fajlovima (<fstream>) kako bi kreirala i zapisala podatke u fajl. Proces uključuje iteraciju kroz matricu lavirinta radi formatiranog ispisa svakog elementa, kao i dodatno zapisivanje informacija o konačnom ishodu igre. Ova funkcionalnost je ključna za pružanje korisniku uvida u završni rezultat igre i trenutni izgled lavirinta.

Klasa UpisStanja dizajnirana je kao statička, što znači da nije potrebno kreiranje instanci ove klase za korišćenje njene funkcionalnosti. Ovaj pristup je praktičan za pomoćne funkcionalnosti poput zapisivanja podataka u fajl, jer eliminiše potrebu za nepotrebnom alokacijom objekata.

You, 4 days ago | 1 author (You)

```
class UpisStanja {  
    public:  
        static void upisiUFajl(Lavirint& lavirint, int stanje);  
};
```

Klasa UpisStanja deklarirana unutar fajla upis_stanja.hpp

4 Tehnička analiza programa

4.1 Ulazna tačka programa – main funkcija

Main funkcija predstavlja ulaznu tačku programa i odgovorna je za inicijalizaciju igre na osnovu ulaznih argumenata, kao i za pokretanje i završavanje igre. Ključne tačke analize uključuju:

- Provera ulaznih argumenata
- Konverzija argumenata u cele brojeve
- Korekcija dimanzije lavirinta
- Validacija vrednosti
- Kreiranje i pokretanje igre
- Završetak funkcije

Main funkcija započinje proverom broja ulaznih argumenata putem uslova `if (argc != 4)`. Ukoliko broj argumenata nije tačno četiri, ispisuje se poruka o pravilnoj upotrebi programa i funkcija se završava vraćanjem vrednosti 1, čime se signalizira greška. Ovo osigurava da korisnik unese tačne parametre pre nego što se igra pokrene.

```
Previous session from Sat Dec 28 at 7:39 PM

~/Desktop/00P2-Minotaur git:(master)±2 (0.03s)
./main_program 15 30
Upotreba: ./main_program <broj_redova> <broj_kolona> <broj_predmeta>

✦ Correct the usage of the main program arguments. Ctrl Shift Enter
```

Argumenti se konvertuju iz stringova (`char*`) u celi brojevi korišćenjem funkcije `std::stoi`. U slučaju da ulazni argumenti nisu validni celi brojevi, hvata se izuzetak (`std::exception`) i ispisuje se odgovarajuća poruka o grešci. Ovo štiti program od prekida u slučaju nevalidnog ulaza. Slika greške na narednoj stranici.

```
~/Desktop/00P2-Minotaur git:(master)±2 (0.029s)
./main_program 15 30 f
Greška: Argumenti moraju biti celi brojevi.
```

✦ Correct the arguments to be integers. Ctrl Shift Enter

Proverava se parnost dimenzija lavirinta (broj redova i kolona). Ukoliko je neka od dimenzija parna, povećava se za 1 kako bi se osigurala neparna veličina lavirinta. Ovo je ključno za generisanje pravilnog lavirinta jer algoritmi za generisanje često zahtevaju neparne dimenzije radi pravilnog rasporeda zidova i prolaza. Moj program koristi **modifikaciju Kruskalovog algoritma**.

Validiraju se minimalne dozvoljene vrednosti za broj redova, kolona i predmeta. Ako broj redova ili kolona nije veći od 15, ili je broj predmeta manji od 3, funkcija ispisuje poruku o grešci i završava se sa povratnom vrednošću 1. Ova validacija osigurava da igra funkcioniše unutar očekivanih parametara.

Ukoliko su svi ulazni parametri validni, kreira se instanca klase Igra sa odgovarajućim parametrima. Poziva se metoda pokreniIgru() koja započinje glavni tok igre. Na kraju se poziva metoda završiIgru() kako bi se igra pravilno završila i oslobodili svi resursi.

Nakon uspešnog pokretanja programa pojavi se kratka poruka o vremenu koje je bilo potrebno generisati lavirint. Pored njega, počinje odbrojavanje tajmera od 3 sekunde. Primer ispod je generisanje lavirinta 51x51.

```
~/Desktop/00P2-Minotaur git:(master)±2
./main_program 50 50 5

Lavirint generisan za 0.000349346 sekundi

Igra počinje za 3 ■
```

```
./main_program 15 60 5
#####U#####
#      P  # # # # #      # # # # #      R  #      #      #
# #####  ###  ###  # # # # #####  # # #  #####  # #####
# #      # # # # #      #      # # # # #      # # #
# ###  #####  # ###  # #####  #####  # ###  #####  #####  # #
# # # # #      P      #      # # # # #      # # # # #
# # # # #  ###  ###  ###  #####  # # # # #  # # #####  # #####
# # # # #      # # # # #      P# # # # # # # #      # # #
###  #####  ###  # # ###  ###  # # #####  # #M###  # ###  # ###  #
# # # # #      # # # # # # # # #      # # # # # # # #
# ###  # # #####  ###  # ###  #####  #####  #####  # ###  # #####
#      # # # # #      # # # # #      # # # # #      # #
# #####  #####  ###  # ###  ###  # #####  ###  ###  # # #####  # #
# #      # # # # #      # # # # #      P  # #P  # # # # #      #
#####I#####
```

Primer generisanja lavirinta 15x51

4.2 Pokretanje igre – funkcija Igra::pokreniIgru()

Funkcija pokreniIgru je centralna metoda klase Igra, koja upravlja celokupnim tokom igre, od generisanja lavirinta, postavljanja elemenata, do procesiranja korisničkih komandi i završetka igre. U nastavku je detaljna analiza ključnih delova funkcije:

- Generisanje lavirinta I inicijalizacija
- Odbrojavanje do početka igre
- Glavna petlja igre
 - Prikaz lavirinta
 - Obrada korisničkih komandi
 - Greške u unosu
- Kretanje minotaura
- Upravljanje efektima
- Završetak igre – upis stanja

Na početku, funkcija poziva lavirint->generisiLavirint(brojPredmeta) kako bi kreirala lavirint sa zadatim brojem predmeta. Potom prolazi kroz matricu lavirinta i pronalazi početne pozicije robota i Minotaura pomoću getSimbol() metode, postavljajući njihove koordinate pomoću setRobotX,

setRobotY, setMinotaurX i setMinotaurY. Ovo osigurava ispravno postavljanje svih elemenata igre.

Implementirano je odbrojavanje od tri sekunde pre početka igre koristeći `std::this_thread::sleep_for` i ispis na konzolu. Ovo dodaje dinamičan efekat za igrača i priprema ga za igru.

Glavna petlja igre funkcionise tako što najpre prikazuje lavirint, koji može biti regularan ili zamagljen u zavisnosti od aktivnog efekta. Prikazivanje lavirinta se realizuje kroz `prikaziLavirint` ili `prikaziLavirintZamagljen`. Igrač unosi komandu putem tastature, a funkcija obrađuje komande poput w, a, s, d za kretanje robota ili q za prekid igre. Na osnovu unosa, robot se pomera pozivom `pomeriRobota`, a trenutni ishod igre procenjuje se metodom `procenaStanja`. Ishodi mogu biti pobeda robota, pobeda Minotaura ili prekid igre. Svaki od ovih ishoda ispisuje odgovarajuću poruku i prekida glavnu petlju.

Ako korisnik unese nevažeću komandu, ispisuje se poruka o grešci i funkcija nastavlja čekanje na validan unos.

Na kraju svake iteracije petlje, Minotaur se nasumično pomera pomoću metode `pomeriMinotauraRandom`. Ukoliko je aktivan neki efekat, broj preostalih poteza efekta se smanjuje pozivom `setTrajanjeEfekta`.

Nakon što igra završi, stanje lavirinta i ishod igre se beleže u fajl korišćenjem statičke metode `UpisStanja::upisiUFajl`.

4.3 Generisanje lavirinta

Funkcija `generisiLavirint` klase `LavirintUtils` odgovorna je za kreiranje i inicijalizaciju lavirinta sa svim potrebnim elementima igre, uključujući zidove, hodnike, ulaz, izlaz, robota, predmete i Minotaura. Funkcija takođe meri vreme potrebno za generisanje lavirinta, pružajući korisniku povratnu informaciju o brzini izvršavanja.

Funkcija započinje merenjem vremena korišćenjem `std::chrono::high_resolution_clock` kako bi se precizno izračunalo trajanje generisanja. Zatim dobija dimenzije lavirinta kroz metode `lavirint.getBrojRedova` i `lavirint.getBrojKolona`. Ove dimenzije se koriste za inicijalizaciju dvodimenzionalnog vektora `vektorLavirint`, koji se koristi za privremeno skladištenje zidova i hodnika lavirinta.

Funkcija generisiVektorLavirint popunjava ovaj vektor inicijalnim vrednostima, nakon čega se vrednosti vektora kopiraju u matricu lavirinta. Svaka ćelija matrice postaje instanca klase Element, pri čemu simbol '#' predstavlja zid, a simbol '.' predstavlja hodnik.

Za dodavanje ulaza i izlaza u lavirint, funkcija nasumično bira kolonu u prvom i poslednjem redu koja nije zid (oznaka 0 u vektoru). Ulaz se postavlja na simbol 'U', a izlaz na simbol 'I'. Ovi elementi se dodaju direktno u matricu lavirinta.

Nakon što su ulaz i izlaz postavljeni, funkcija poziva dodatne metode postaviRobota, postaviPredmete i postaviMinotaura kako bi se u lavirint dodali ključni elementi igre. Ove metode osiguravaju pravilno postavljanje elemenata na validne pozicije u lavirintu. Na kraju, funkcija ponovo meri vreme i izračunava trajanje generisanja lavirinta. Korisniku se ispisuje poruka o vremenu potrebnom za generisanje korišćenjem std::cout.

4.4 Algoritam generisanja lavirinta

Funkcija generisiVektorLavirint generiše osnovnu strukturu lavirinta kao dvodimenzionalni vektor, pri čemu vrednost 1 predstavlja zid, a vrednost 0 hodnik. Ova funkcija implementira algoritam za generisanje lavirinta zasnovan na Kruskalovom algoritmu za minimalno razapeto stablo, koristeći strukturu disjunktih skupova za povezivanje ćelija.

Funkcija započinje inicijalizacijom dimenzija lavirinta i generatora nasumičnih brojeva korišćenjem srand(time(0)). Zatim se kreira vektor ivice, koji sadrži sve moguće grane između ćelija lavirinta. Grane su definisane kao parovi koordinata između susednih ćelija. Ove grane se generišu samo za neparne indekse ćelija, kako bi se očuvala struktura lavirinta.

Nakon što su sve grane dodate u vektor ivice, one se nasumično permutuju korišćenjem std::swap i nasumičnih indeksa generisanih pomoću rand(). Ovo obezbeđuje da se hodnici lavirinta generišu na nasumičan način.

Struktura disjunktih skupova, implementirana u klasi DisjunktniSkup, koristi se za praćenje povezanosti ćelija. Svaka ćelija se na početku postavlja u zaseban skup. Vrednost 1 se dodeljuje svim ćelijama lavirinta, označavajući da su sve ćelije u početku zidovi.

Funkcija prolazi kroz sve grane u vektoru ivice. Za svaku granu proverava da li ćelije koje povezuje pripadaju različitim skupovima. Ako pripadaju, grana se dodaje u lavirint tako što se ćelijama koje povezuje dodeljuje vrednost 0, čime se kreira hodnik. Istovremeno, grana između ove dve ćelije takođe postaje hodnik. Ova provera i spajanje skupova obezbeđuju da lavirint ostane povezan, bez stvaranja ciklusa. Rezultat ove funkcije je vektor lavirint, koji predstavlja osnovnu matricu lavirinta sa nasumično generisanim hodnicima i zidovima.

4.5 Struktura ivica I disjunktni skup

Struktura Ivica definiše jednu ivicu u lavirintu koja povezuje dve ćelije. Sastoji se od koordinata dve ćelije, x_1 i y_1 za prvu ćeliju, i x_2 i y_2 za drugu ćeliju. Ova struktura se koristi za modelovanje mogućih veza između ćelija lavirinta tokom njegovog generisanja.

Konstruktor klase DisjunktniSkup inicijalizuje strukturu disjunktnih skupova za zadati broj elemenata, označen parametrom velicina. Vektor roditelj se inicijalizuje tako da svaki element bude svoj sopstveni roditelj, čime svaki element započinje u zasebnom skupu. Vektor rang se inicijalizuje na nulu za sve elemente, što označava da svi skupovi u početku imaju dubinu 0.

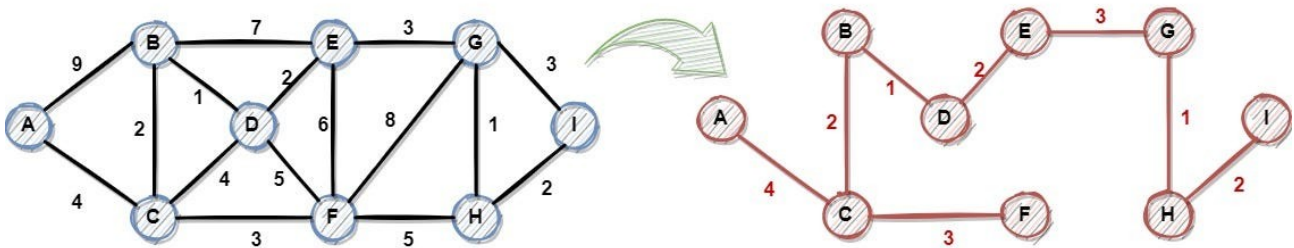
4.6 Spajanje disjunktnih skupova

Metoda spojiSkupove klase DisjunktniSkup implementira spajanje dva skupa pomoću optimizacije union by rank. Cilj ove metode je efikasno spojiti dva skupa elemenata, dok se održava minimalna dubina strukture.

Metoda započinje pronalaženjem korena skupova kojima pripadaju elementi x i y pozivanjem metode pronadji. Ako koreni oba elementa već pripadaju istom skupu, metoda vraća false, jer spajanje nije potrebno.

U slučaju da elementi pripadaju različitim skupovima, metoda koristi rang za određivanje koji koren postaje roditelj drugog. Ako je rang korena x manji od ranga korena y , roditelj korena x postaje koren y . Ako je rang korena x veći, roditelj korena y postaje koren x . Ako su rangi jednaki, koren y postaje potomak korena x , a rang korena x se povećava za jedan. Na kraju, metoda vraća true, što označava da je spajanje uspešno obavljeno. Više o Kruskal algoritmu: https://en.wikipedia.org/wiki/Kruskal%27s_algorithm

Kruskal's Algorithm



4.7 Kretanje igrača – funkcija Igra::pomeriRobota(...)

Metoda `pomeriRobota` implementira logiku pomeranja robota u lavirintu na osnovu zadatih koordinata x i y . Ova metoda upravlja interakcijama robota sa različitim elementima u lavirintu, uključujući hodnike, zidove, ulaz, izlaz, predmete i Minotaura.

Prva provera je za slučaj da robot stigne do ćelije označene simbolom 'I', što označava izlaz. U tom slučaju, postavlja se da je robot pobedio, igra se završava, i trenutna pozicija robota se oslobađa zamenom sa praznim elementom. Zatim se ažuriraju koordinate robota.

Ako robot pokuša da uđe u ćeliju sa zidom označenim simbolom '#', proverava se trajanje efekata. Ako efekat nije aktivan, ili ako nije aktiviran specifičan efekat "Čekić", kretanje se blokira. Ako je efekat "Čekić" aktivan i zid nije na granici lavirinta, zid se uklanja, a robot se pomera na zadate koordinate.

Ako robot pokuša da se vrati na ulaz označen simbolom 'U', kretanje je blokirano.

Kada robot naiđe na ćeliju sa Minotaurom označenim simbolom 'M', ishod zavisi od trenutnog efekta. Ako je aktivan efekat "Mač", Minotaur se uklanja, a robot zauzima njegovo mesto. Ako je aktivan efekat "Štit", robot ostaje na trenutnoj poziciji. U svim drugim slučajevima, Minotaur pobeđuje, a igra se završava.

Ako robot naiđe na ćeliju sa predmetom označenim simbolom 'P' (*), efekat predmeta se aktivira na tri poteza, ažurira se trenutni efekat, a robot preuzima ćeliju sa predmetom. Konačno, ako robot naiđe na praznu ćeliju označenu simbolom ' ', robot se jednostavno pomera na zadate koordinate, dok se prethodna pozicija robota oslobađa.

(*) - kada robot stane na efekat, dolazi do aktivacije efekta instant, a pošto minotaur igra odmah posle robota, deluje kao da robot ima samo 2 runde da koristi efekat, a zapravo koristi tri runde. Problem: šta ako je robot stao na efekat 'Mač', minotaur igra posle njega I igrom slučaja nalazi se na jedno polje od njega, pa mora da ga pojede. Bilo bi pogrešno da je robot upravo pokupio mač a da ne može da ga iskoristi na minotauru. Zbog toga ako se takva situacija desi, minotaur odmah gubi život, I to se računa kao jedna runda. Naravno ista situacija je I sa štitom. Medjutim stvari se menjaju kod magle rata I čekića... Magla rata će naizgled trajati '2.5' runde a čekić praktično 2 runde za upotrebu. Znači svaki efekat traje podjednako dugo, u zavisnosti od konteksta deluje kao da nije iskorišćen potpuni potencijal efekta.

4.8 Kretanje minotaura

Metoda pomeriMinotauraRandom implementira nasumično kretanje Minotaura unutar lavirinta. Funkcija proverava trenutne okolnosti, uključujući status Minotaura, položaj robota i eventualne prepreke, i prilagođava ponašanje Minotaura u skladu sa tim.

Funkcija započinje proverom da li je Minotaur živ, pomoću metode getMinotaurZiv. Ako Minotaur nije živ, metoda se odmah prekida. Nakon što se pribave trenutne koordinate Minotaura pomoću getMinotaurX i getMinotaurY, kreira se niz potencijalnih smerova kretanja (gore, dole, levo, desno), koji se nasumično mešaju koristeći std::shuffle i std::default_random_engine za generisanje nasumičnih brojeva.

Ako je robot u neposrednoj blizini Minotaura, funkcija proverava aktivne efekte. Ako je aktivan efekat "Mač", Minotaur se uklanja iz igre, a njegovo mesto zauzima prazna ćelija (zaletio se u robota I poginuo je). Ako je aktivan efekat "Štit", Minotaur 'ne napada' (zapravo napada samo ništa ne učini) i ostaje na trenutnoj poziciji. U svim ostalim slučajevima, Minotaur pobeđuje, igra se završava, a Minotaur zauzima poziciju robota.

U slučaju da robot nije u blizini, Minotaur se nasumično pomera u jednom od dostupnih pravaca. Funkcija iterira kroz nasumično pomešane smerove i proverava da li je ciljna ćelija prazna ili sadrži predmet. Ako je prazna, Minotaur se pomera na tu poziciju, a prethodna ćelija postaje prazna.

Ako ciljna ćelija sadrži predmet, predmet se uništava, a Minotaur zauzima tu ćeliju.

4.9 Upis stanja

Metoda upisiUFajl klase UpisStanja omogućava zapisivanje trenutnog stanja igre u fajl. Ova funkcija beleži ishod igre i trenutni izgled lavirinta, čuvajući podatke u tekstualnom fajlu pod nazivom "lavirint.txt".

Metoda započinje otvaranjem fajla koristeći `std::ofstream`. Ukoliko je ishod igre poznat, ispisuje odgovarajuću poruku na osnovu vrednosti parametra stanje. Vrednost 1 označava pobedu robota, 2 pobedu Minotaura, dok 3 označava prekid igre. Nakon toga, dodaje prazan red radi razdvajanja poruke i matrice lavirinta.

Funkcija zatim prolazi kroz matricu lavirinta, koristeći metode `lavirint.getMatrica`, `lavirint.getBrojRedova` i `lavirint.getBrojKolona` kako bi iterirala kroz sve ćelije. Za svaku ćeliju, zapisuje njen simbol u fajl pomoću metode `getSimbol`. Na kraju svakog reda, dodaje novi red u fajl. Kada je zapisivanje završeno, fajl se zatvara pozivom metode `close`, čime se osigurava pravilno čuvanje podataka.

4.10 Završi igru I oslobodi resurse

Metoda `završiIgru` u klasi `Igra` i destruktor klase `Lavirint` zajedno upravljaju pravilnim oslobađanjem resursa korišćenih tokom igre. Njihova funkcija je da osiguraju efikasno upravljanje memorijom i spreče curenje resursa na kraju igre.

Metoda `završiIgru` započinje proces uništavanja lavirinta tako što poziva operator `delete` na pokazivaču `lavirint`, koji je atribut klase `Igra`. Ovo automatski poziva destruktor klase `Lavirint`, jer se oslobađa dinamički alocirani objekat na koji pokazuje `lavirint`. Nakon toga, pokazivač `lavirint` se postavlja na `nullptr` kako bi se obezbedilo da ne pokazuje na nevalidnu memorijsku lokaciju.

Destruktor klase `Lavirint` prolazi kroz matricu elemenata lavirinta. Za svaku ćeliju matrice poziva se operator `delete` kako bi se oslobodila dinamički alocirana memorija za objekte unutar ćelije. Nakon oslobađanja svih ćelija, oslobađaju se i redovi matrice korišćenjem `delete[] matrica[i]`, a zatim i sama matrica korišćenjem `delete[] matrica`.

4.11 Nasledjivanje I polimorfizam

Polimorfizam i nasleđivanje korišćeni su u ovoj igri za rad sa različitim elementima lavirinta. Klasa Element predstavlja osnovnu klasu, dok klase Robot, Minotaur i Predmet nasleđuju ovu klasu i proširuju je dodatnim funkcionalnostima. Polimorfizam se ogleda u tome što se objekti ovih izvedenih klasa mogu tretirati kao objekti osnovne klase Element, omogućavajući uniformno upravljanje svim elementima lavirinta. Na primer, metoda getSimbol omogućava dinamičko određivanje simbola objekta u zavisnosti od njegove konkretne klase.

Polimorfizam je takođe iskorišćen u dinamičkom kastovanju, poput `dynamic_cast`, za konverziju objekata klase `Element` u specifične izvedene klase, kao što je `Predmet`. Ovo se koristi za pristup specifičnim metodama ili atributima izvedenih klasa, poput aktivacije efekta predmeta.

[illegible]

```
#####U#####
#           #           #
#####   #   #   #   #   #   #
#           #   #   #           #
###   ###   ###   #   ###   #   #####
#   #           #   #           #   #
#####   #####   ###   ###   #
#           #           #           #
#   #####   #   #####   ###   ###   #
#   #           M           #   #   #   #
###   #   ###   ###   #   ###   #   ###
#   #P   #           #           #
#   ###   #   #   #P   #           #
#   PP   #   #   #           #           #
#####I#####
```

Efekat Magla rata traje još 2 runde

Minotaur je pobedio!

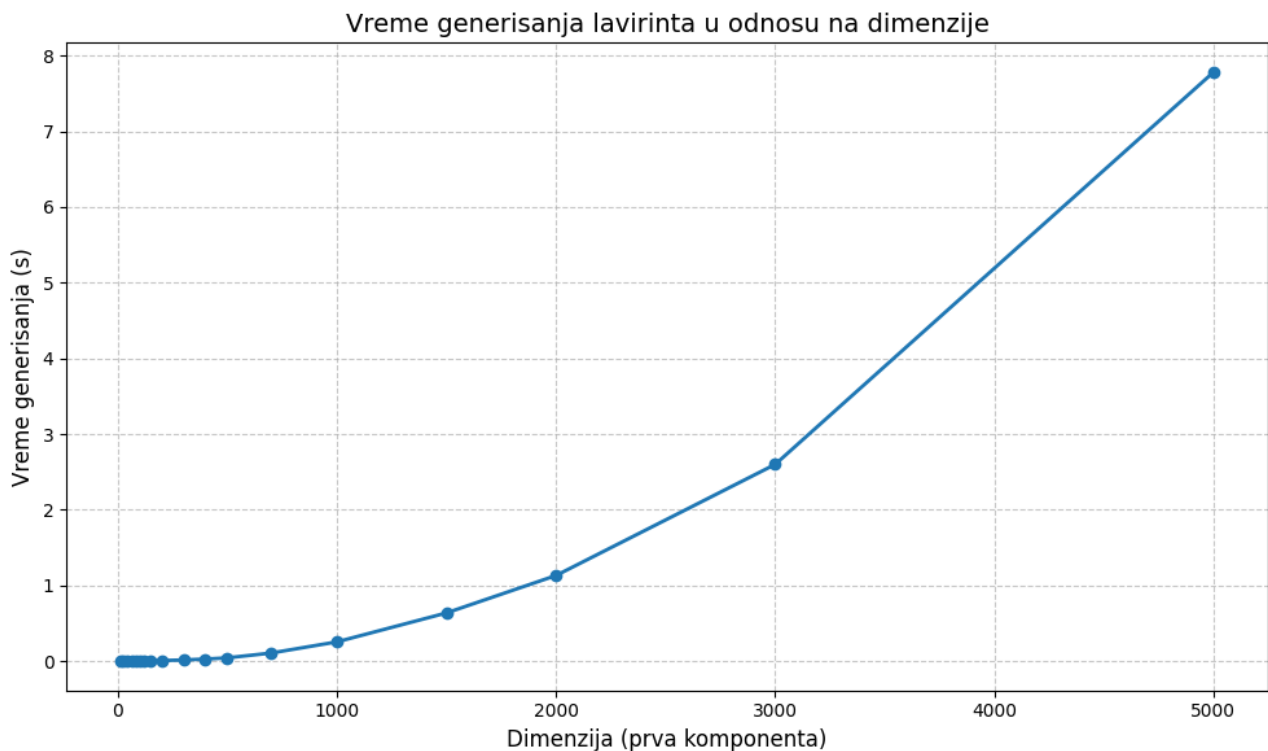
```
#####U#####
#      #      #      #      #
# # # # ##### #
# # #      #      #      #
### ##### # # ### # # #
# #      #      #      # # # #
# ### # # # # ###P### ###
#      #      #      #      #
# # # # ##### # ##### #
# #      #P      #      #
### ##### ### # # ###
#      # # # #      # # # #
# ##### # # ### # ### # #
#      #      PP      #      M#
#####I#####
```

```
#####U#####
#  #                                     R#  #
# ##### ##### ##### #
#  #  #  #  #  #  #  #
# # ### # ##### # # # ###
# # # #  # #  # # # #
# ### # ### ### # # # # #
#      # # # #      # # #
# # ### # # ##### ### ###
# #P      #  # P  #  #
##### #####P### # ### # #
#  #                P# # #M# # #
# # # ##### ##### # # # #
#P#      #  #  #  #  #
#####I#####
```

Robot je pobedio!

Igra je prekinuta! (Q)

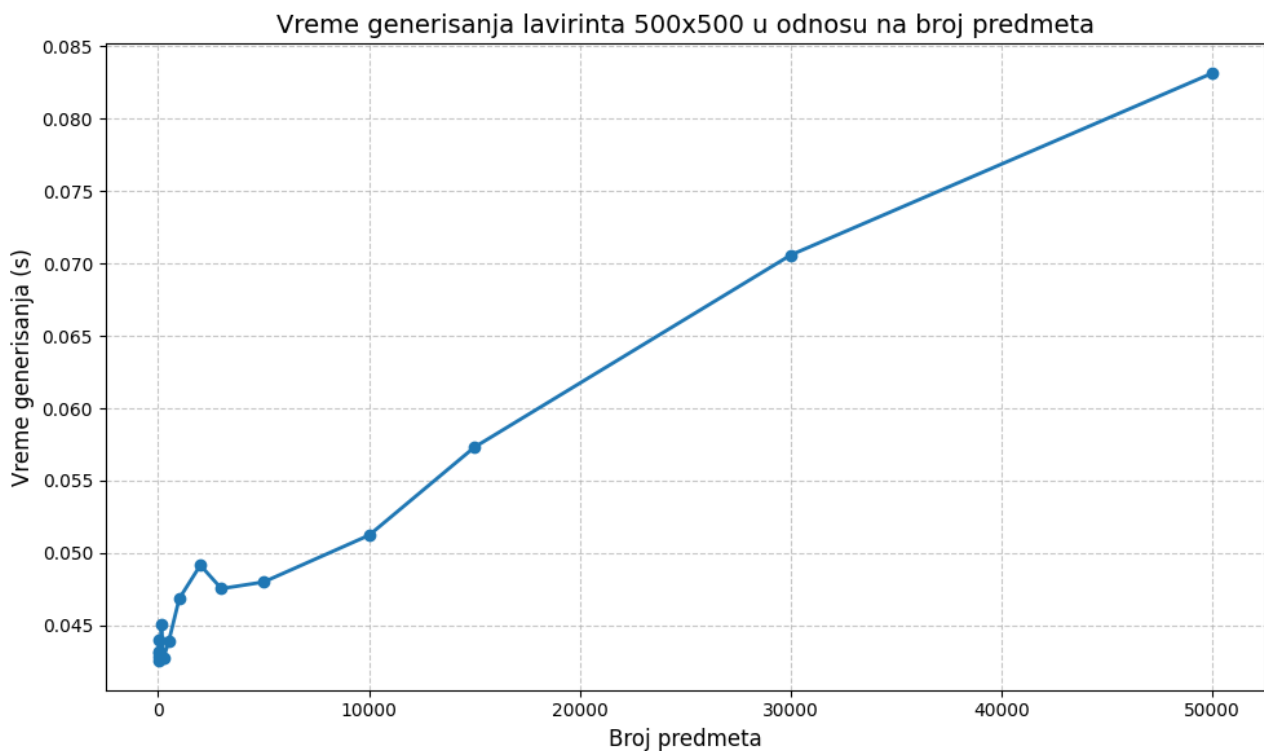
5 Testiranje vremena generisanja



Na grafiku je prikazano potrebno vreme generisanja lavirinta sa 5 predmeta kvadratne dimenzije ($A \times A$).

```
dimensions = [  
    "15x15", "25x35", "45x45", "65x65", "85x85", "100x100",  
    "120x120", "150x150", "200x200", "300x300", "400x400",  
    "500x500", "700x700", "1000x1000", "1500x1500",  
    "2000x2000", "3000x3000", "5000x5000"  
]  
times = [  
    4.837e-05, 0.00013071, 0.000281942, 0.000564675, 0.000996664,  
    0.00143623, 0.00200047, 0.00320132, 0.0058335, 0.013609,  
    0.0255726, 0.0416975, 0.104973, 0.252599, 0.635317,  
    1.12995, 2.59846, 7.78192  
]
```

Sledeći grafik prikazuje potrebno vreme generisanja lavirinta ako broj predmeta nije fiksno postavljen na 5. Uzećemo lavirint dimenzija 500 x 500 i postepeno povećavamo broj predmeta. Na malim dimenzijama broj predmeta zaista gotovo da nema nikakvog uticaja... Slika grafika je na sledećoj strani.



Jasno je da prema grafiku koji ima $500 \times 500 = 250000$ ćelija, kada broj ćelija koje su predmeti predje 4% dolazi do postepenog povećanja vremena, sve pre toga je otprilike slično vreme potrebno za generisanje.

```
# Podaci za dimenziju 500x500 i promenljiv broj predmeta
num_items = [
    5, 10, 20, 30, 50, 100, 150, 250, 500, 1000,
    2000, 3000, 5000, 10000, 15000, 30000, 50000
]
times = [
    0.0431699, 0.0425756, 0.0431082, 0.04287, 0.043984,
    0.0429525, 0.0451087, 0.0427403, 0.0438892, 0.0468571,
    0.0491621, 0.0475462, 0.0480014, 0.0512306, 0.0573104,
    0.0705785, 0.0831376
]
```

Kod ekstremnih slučajeva, poput 50000 i više predmeta, primetno je značajno povećanje vremena, što je očekivano s obzirom na količinu dodatnih provera i interakcija sa strukturom lavirinta.

Prvi grafikon, koji prikazuje vreme generisanja lavirinta u zavisnosti od dimenzija, podseća na eksponencijalnu funkciju $y=a \cdot e^{bx}$, jer vreme generisanja raste sve brže kako se dimenzije povećavaju.

Drugi grafikon, koji pokazuje vreme generisanja lavirinta dimenzija 500x500 u zavisnosti od broja predmeta, ima oblik približno linearnog rasta sa povremenim fluktuacijama, što podseća na linearnu funkciju $y=mx+c$, posebno u delu nakon prelaska praga od 10000 predmeta.

6 Izlazni rezultati – fajl lavirint.txt

Fajl lavirint.txt predstavlja završni zapis trenutnog stanja igre nakon njenog završetka. Sadrži ishod igre, poput poruke "Robot je pobedio!" ili "Minotaur je pobedio!", što pruža jasan zaključak za igrača. Nakon ove poruke, fajl uključuje matricu lavirinta, gde svaki simbol ('#' za zidove, '.' za hodnike, 'R' za robota, 'M' za Minotaura, itd.) precizno oslikava završnu strukturu lavirinta.

```
You, 1 second ago | 1 author (You)
Igra je prekinuta (Q)!

####U#####
# R  # # #  # #  # # P  # # #  # #  # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # #  # # #  # # #  # # #  # # #  # # #  # # #  # #
# ##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
##### # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # #
#####I#####
```

Primer lavirinta 25x90 sa 10 predmeta